

“Specialization” of a dictionary may be appropriate when it is used to compress specialized data. However, we should caution the reader to note that all one can really claim about the statistics reported in Appendices A.1.1 through A.1.9 is that they reflect the files listed in Appendix A.1, give some loose indication about technical writing in computer science, and only a slight indication of English writing as a whole. In order to get accurate statistics about a source such as written English, even in a restricted environment such as computer science technical writing, much more data would have to be sampled (i.e., by at least several orders of magnitude over the sample size of 1 million characters used here).

1.4. COMPRESSION BY TEXTUAL SUBSTITUTION

A powerful (and practical) approach to compressing a string of characters is *textual substitution*. A textual substitution data compression method is any compression method that compresses text by identifying repeated substrings and replacing some substrings by references to other copies. We call such a reference a *pointer*[†] and the string to which the pointer refers the *target* of the pointer. Thus, the input to a data compression algorithm employing textual substitution is a sequence of characters over some alphabet Σ and the output is a sequence of characters from Σ interspersed with pointers. We make the distinction between pointers and characters only for convenience. Since any well defined compression algorithm must be able to take as input a string consisting of a single character, it must be possible to represent any single character with whatever coding scheme is used to represent the output from the algorithm. Hence, a character can be viewed as a pointer to a string of exactly one character and the output of the compression algorithm as simply a sequence of pointers, where each pointer specifies some string of ≥ 1 characters of Σ .

[†] In the literature, such references are often called *codewords* or *tokens*.

1.5. OFF-LINE VERSUS ON-LINE ALGORITHMS

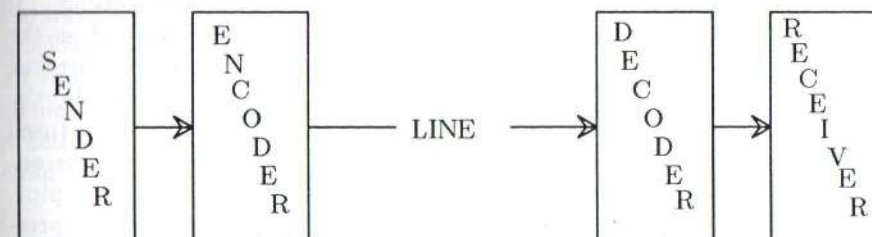


Figure 1.5.1

As depicted by Figure 1.5.1, a practical model for *on-line* data compression by textual substitution is to have an *encoder* and *decoder*, each with a fixed (finite) amount of local memory, which we refer to as a *local dictionary*, D . We assume that the two local dictionaries may be initialized at the beginning of time to contain identical information. The encoder connects the *communication line* to the *sender* and the decoder connects the communication line to the *receiver*. What distinguishes this model from an *off-line* model is that neither the sender nor the receiver can see all of the data at once; data must be constantly passing from the sender through the encoder, through the decoder, and on to the receiver. In theory, if D is large, then the encoder, for example, could read in as many bits of data as D is capable of storing (possibly more if the data is compressible), before having to send any bits to the receiver. However, in practice, the algorithms to be presented in this chapter can easily be implemented to run in real-time.

Definition 1.5.1: A compression-decompression method is *real-time* if there exists a constant k (which does not depend on the data being processed) such that for every k units of time, exactly one new character is read by the encoder and exactly one character is written by the decoder. The only exception to this rule is that we may allow a small “lag” between the encoder and decoder. That is, there is another constant l (that is independent of the data being processed) such that during the first l units of time the decoder may produce no characters and, in the case that the input is finite in length, l

time units may pass between the time that the encoder has finished reading characters and the decoder is finished outputting characters. ○

In Section 1.1, data storage and data communications were mentioned as the two most common applications of data compression. If compression algorithms are on-line, then data storage can be viewed as the special case of data communications where the sender and receiver are the same and the communication link is the storage device. For example, the "compact" command that is available on the Berkeley UNIX system (used primarily for compressing files contained in secondary storage) employs on-line Huffman coding; this allows it to be used as part of a command pipe. There is a subtle distinction, however, when one considers off-line compression. That is, it is typical with data storage to think of storing files of given sizes whereas with communications, one often does not know in advance how long the transmission will last. Since most of the practical algorithms that we shall consider are on-line, we shall typically focus our examples and informal discussions on data communications. It should be noted, however, that from a theoretical point of view, off-line methods are more powerful than on-line methods for compressing finite length strings under certain models. This issue will be considered in Chapters 5 and 6.

For the purposes of this book, we shall assume that the dictionary D of an on-line data compression algorithm is stored in standard random access memory (RAM memory), that is both readable and writable, and that integer variables used in our algorithms can be large enough to index all memory locations that are used to store D . The major exception to this assumption will be Chapter 4 where direct VLSI implementations of D are considered. Throughout this book, the following notation pertaining to strings and dictionaries will be used: For an integer i , we use the standard notation that $|i|$ denotes the absolute value of i (i if $i \geq 0$ or $-i$ if $i < 0$). However, we also use vertical bars for other purposes. For a string s , $|s|$ denotes its length. For a set S , $|S|$ denotes its cardinality. Thus, $|D|$ refers to the current number of entries that D contains. In addition, $\langle D \rangle$ refers to the maximum number of entries that D may contain. Note that, in general, entries of D are variable length character strings, and it may be that $|D|$ and $\langle D \rangle$ are quite different than the amount of memory needed to store D ; this issue shall be addressed shortly. From this point on we shall always assume $\langle D \rangle$ to be a power of 2. Although this assumption is not necessary, it simplifies notation and is almost always the case in practice[†].

[†] Otherwise, $\log_2 \langle D \rangle$ is not an integer and a number of bits equal to the least integer greater than this value will have to be used to index values of D (at least when D is full); this "round-off error" must necessarily result in some unused bit sequences.

A data structure that allows us to efficiently store a dictionary of strings is the following. Given an alphabet Σ , a *trie* is a tree where edges are labeled by elements of Σ in such a way that children of a given parent are connected via edges that have distinct labels, all leaf nodes are labeled as "marked", and all internal nodes are labeled as either "marked" or "unmarked". The set of *strings represented by a trie* are those that correspond to all root to marked node paths. Although we shall continue to talk about an element of Σ being "associated with an edge of the trie", it is convenient in practice to store the element of Σ that is associated with a given edge in the child attached to that edge.

Example 1.5.1 (A Trie): For the trie in Figure 1.5.2, the marked nodes are indicated by boxes and the unmarked nodes by circles. That is, this trie stores the strings "bear", "beaver", "bee", "bird", "cat", "cattle", "cow", "doe", and "dog". ○

Since leaves of a trie are always marked, a data structure need only associate a mark bit with each non-leaf node of the trie. In addition, there are many practical applications where all or none of the internal nodes are considered in the set of strings represented by the trie; in both of these extremes, mark bits are not needed at all.

There are several standard ways to represent a trie (which have different time-space tradeoffs). One way is to store with each node an array of length $|\Sigma|$ that stores the locations of the children of that node; this representation gives constant access time but can be wasteful of space if the average degree of a node is small with respect to $|\Sigma|$ (often the case in practice). A second way is to store with each node a linked list of its children; this representation is space efficient but can be time consuming if the average degree of a node is large. A third way is to have a global hash table that, when given a node and an element of Σ , returns the corresponding child; this representation is space efficient and gives constant expected access time. The bibliographic notes contain references to introductory material on the trie data structure as well as other tree data structures.

Independent of whether a trie or some other data structure is used to represent a dictionary, $\langle D \rangle$ may be much different than the actual number of bytes[†] used to represent D . Consider the following two examples:

[†] As is standard in the literature, when discussing amounts of computer memory, we shall use units of *bytes* (8 bits) or *kilo-bytes* ($2^{10}=1024$ bytes), which we shall abbreviate with a K (e.g., $3K$ bytes = 3,072 bytes).

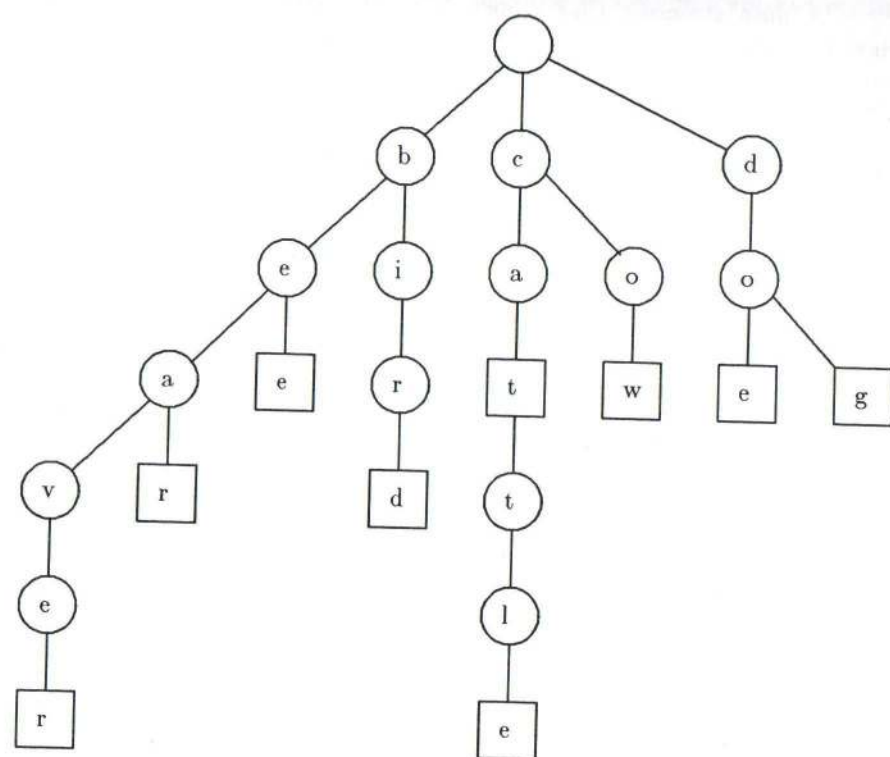


Figure 1.5.2
A Trie

Example 1.5.2: A dictionary of $2^{12}=4,096$ entries, each entry being at most 8 characters long is stored as follows. Each character is represented by a byte and each entry is always represented by 9 bytes; the first byte contains the length of the entry. Entries are stored in lexicographic order in a 4,096 element array (9 bytes per element) so binary search can be used to locate entries. This representation uses 9 bytes per entry for a total of $9 \times 4K = 36K$ bytes. ○

Example 1.5.3: A dictionary of $2^{16}=65,536$ entries is to be stored. There is no limit (except for the size of D) on how long each entry can be, however, it is the case that whenever an entry is stored, so are all of its prefixes. The entries are stored in a trie data structure where children of a given node are stored in a linked list; each node of the trie contains the fields:

char:	1 byte, character corresponding to this node
lchild:	2 bytes, left-child pointer
rsib:	2 bytes, right-sibling pointer
parent:	2 bytes, parent pointer

Because all prefixes of a given entry must also be in D , each entry of D consumes one new trie node of 7 bytes, for a total of $7 \times 64K = 448K$ bytes. ○

For a given implementation I of a dictionary D , we let $\langle\langle D, I \rangle\rangle$ denote the number of bits used to represent D with the implementation I ; when I is understood, we simply write $\langle\langle D \rangle\rangle$. In practice, the designer is often given a maximum value for $\langle\langle D, I \rangle\rangle$ and then asked to produce data compression software or hardware. We shall see in the next section that there can often be tradeoffs between how fast the algorithm runs, how much space it uses, and how well it compresses data. For example, if more memory per dictionary element is needed for a faster algorithm, then the value of $\langle D \rangle$ must be made smaller (if the total amount of memory used is to remain the same), which in turn could degrade the amount of compression achieved. The size of the program itself must also be counted. However, for the methods that we shall consider, the difference in program size from one implementation to another will not usually be significant. In addition, when hardware is being built, the program can occupy read-only memory (ROM memory), whereas D will require the usual readable and writable random access memory for most of the methods we consider[†].

[†] We have avoided a few small details. In particular, although we shall not consider programs that "cheat" by encoding large amounts of memory into the program code itself, we do allow programs to have a constant number of integer variables (for indexing, to point into data structures, etc.) which we have not formally included in the definition of $\langle\langle D, I \rangle\rangle$. In practice, we assume that $\langle\langle D, I \rangle\rangle$ is large enough to accommodate a small amount of extra space for such program variables (say 100 extra bytes).

1.6. BIBLIOGRAPHIC NOTES

For introductory and survey material on data compression see Lelewer and Hirschberg [1987], Lynch [1985], Sedgewick [1983], Held [1983], Cooper and Lynch [1982], McEliece [1977], Davisson and Gray [1976], and Lesk [1970].

Bennett [1976] contains an interesting informal introduction to information theory, as does Miller [1951]. For a more thorough introduction to entropy, the reader may refer to a host of publications such as the original paper by Shannon (in Shannon and Weaver [1949]) or the books of Hamming [1980], Ingels [1971], Gallager [1968], Ash [1965], and Abrahamson [1963]. See Gallager [1968] for a discussion of ergodic sources.

There has been much work concerning the application of entropy to more general models of communication. For example, Shannon [1959], Posner and Rodemich [1971], and many others consider channels with an associated fidelity. In addition, Gamal and Orlitsky [1984] consider the exchange data over a two-way communication channel and Wolfowitz [1960] considers the situation when the distribution of error is known by only one of the sender and receiver. The book by Berger [1971] is devoted entirely to rate distortion theory. The book edited by Slepian [1974] is a collection of important early work in information theory. See also Jelinek [1968]. Shannon [1951] estimates the entropy of printed English.

Eichelberger, Rodgers, and Stacy [1968] discuss optimizing the speed of a printer by arrangement and duplication of characters on the print head.

The statistics reported in Appendix A.1 were gathered by Storer and Tsang [1984]. Pike [1981] tabulates word frequencies in English text. Suen [1979] considers n -gram statistics for natural language processing. Yanakoudakis, Goyal, and Huggil [1982] tabulate n -gram frequencies occurring in a large number of bibliographic records; unlike many such tabulations, their statistics do cross word boundaries (but they map upper to lower case). Schwartz and Kleiboemer [1967] has references to studies on the entropy of English. For other statistics on written English, see the book by Kucera and Francis [1967]. Note that the statistics reported in their book are of a different flavor than those reported in Appendix A.1. In Appendix A.1, no attempt is made to attach any meaning to the text, and substring frequencies are reported without any regard to word boundaries, etc. Kucera and Francis [1967] is primarily concerned with the frequencies of basic "pieces" of English, such as word frequencies; their statistics do not take into account upper and

lower case letters, punctuation, or the space and newline characters. However, these statistics do provide insights into the structure of English text that are not apparent from Appendix A.1. For example, from the statistics reported in their book, it follows that for technical writing, approximately 45 percent of all text consists of the 100 most common words, distributed roughly as follows:

2 percent	1 letter words
17 percent	2 letter words
17 percent	3 letter words
6 percent	4 letter words
3 percent	5 or more letter words

Miller [1951] claims that the 50 most common English word types make up 45 percent of those written and 60 percent of those spoken. Douchette, Harrison, and Schuegraf [1977] consider compressing French, English, and German.

Zipf's law is a statistical observation about word frequencies in English text. It says that if the frequency of occurrence of each word is plotted against its rank in a list of all words that goes from most common to least common, then a relatively straight line results. That is, if f_i denotes the frequency of the i^{th} word in this list, then f_i is roughly proportional to $1/i$. Chapter 5 of Young [1971] and Chapter 4 of Miller [1951] contain a presentation of Zipf's law. See also Schwartz and Kleiboemer [1967] and Zipf [1935, 1949].

Numerical data contained in databases, spread-sheets, etc. is typically non-uniformly distributed. It is somewhat surprising that this is also true of numerical data relating to physical constants arising in nature. For example, the book of Knuth [1969] contains a discussion of why such constants are likely to have small values for the first digit (e.g., for tables of physical constants written in base-10, 1 is the first digit about 30 percent of the time). For other references to this issue see: Raimi [1976, 1969, 1969b], Mardia [1972], Flehinger [1966], Konheim [1965], Pinkham [1961], Furry and Hurwitz [1945], Benford [1938], and Newcomb [1881].

For introductory material on data structures, including graphs and the trie data structure, see Aho, Hopcroft, and Ullman [1983] or Sedgewick [1985].